

Dzień 26 - NTP, logi, SNMP i monitoring

Szczegółowy materiał do nauki Junior Network Engineer - teoria, zastosowanie w firmie, laboratorium, diagnostyka, bezpieczeństwo i skrypt Bash

Status: część obowiązkowa Dnia 26 została wykonana praktycznie i zweryfikowana. W dokumencie wyraźnie rozdzielono to, co zostało wykonane w laboratorium, od rozszerzeń produkcyjnych.

Spis treści

- 1 Co obejmuje Dzień 26 i po co te technologie są potrzebne
- 2 NTP i synchronizacja czasu - zastosowanie w firmie
- 3 NTP - wykonane laboratorium i interpretacja wyników
- 4 Logi: journald, journalctl i rsyslog
- 5 Centralny syslog w firmie
- 6 SNMP - manager, agent, MIB, OID i wersje protokołu
- 7 SNMP - wykonane laboratorium krok po kroku
- 8 Monitoring interfejsów i liczników ruchu
- 9 Bezpieczeństwo SNMP w środowisku firmowym
- 10 Skrypt diagnostyczny Bash - pełne omówienie
- 11 Praktyczny schemat diagnostyczny juniora
- 12 Bonus: rzeczywista diagnostyka RDP/VM z początku sesji
- 13 REST API, JSON, Ansible, Terraform i AI - krótkie wprowadzenie
- 14 Co warto umieć powiedzieć na rozmowie rekrutacyjnej
- 15 Checklisty, rollback i następne kroki

1. Po co te technologie są potrzebne w firmie?

Dzień 26 łączy cztery obszary, które są bardzo częste w pracy administratora i junior network engineer: czas, logi, monitoring i automatyczną diagnostykę. Nie są to osobne wyspy. Razem pozwalają szybciej zauważyć awarię, ustalić jej przyczynę i udokumentować przebieg zdarzeń.

Technologia	Do czego służy	Przykład w firmie
NTP	Utrzymuje zgodny czas na urządzeniach	Porównanie czasu awarii routera, firewalla i serwera
Syslog / journald / rsyslog	Zbiera i przechowuje zdarzenia	Centralny serwer logów dla routerów, serwerów i VPN
SNMP	Odczytuje stan i liczniki urządzeń	UP/DOWN portów, ruch RX/TX, uptime, alarmy
Skrypt Bash	Automatyzuje powtarzalne testy	Jedno polecenie sprawdza IP, bramę, DNS, NTP i usługi

Najważniejsza zależność:

```
Awaria
|
+--> monitoring wykrywa zmianę
|
+--> logi pokazują zdarzenie
|
+--> NTP zapewnia prawidłowy czas
|
+--> skrypt diagnostyczny szybko sprawdza podstawy
|
`--> administrator zawęży przyczynę
```

2. NTP - po co synchronizuje się czas w firmie?

NTP (Network Time Protocol) służy do utrzymywania zgodnego czasu na urządzeniach. To pozornie prosty temat, ale ma ogromne znaczenie podczas awarii i w systemach uwierzytelniania.

Najważniejsze zastosowania biznesowe:

- Korelacja logów: jeżeli firewall, router i serwer mają różny czas, trudno ustalić kolejność zdarzeń.
- Active Directory i Kerberos: duże rozbieżności czasu mogą powodować problemy z uwierzytelnianiem.
- Certyfikaty: ich ważność jest zależna od czasu. Błędny zegar może powodować błędne komunikaty o nieważnym certyfikacie.
- Zadania cykliczne: backupy, cron, automatyzacja, rotacja logów i harmonogramy wymagają poprawnego czasu.
- Audyt i bezpieczeństwo: analiza incydentu bez spójnej osi czasu jest dużo trudniejsza.

Przykład problemu bez synchronizacji czasu:

```
Router: 12:01:30 utrata WAN
Firewall: 11:58:40 zerwany VPN
Serwer: 12:06:15 błąd aplikacji
```

Bez zgodnego czasu trudno ustalić, co było pierwszą przyczyną.

W poprawnie zsynchronizowanym środowisku:

```
źródło czasu
|
+-----+-----+
| | |
Router Firewall Serwer
| | |
log 1 log 2 log 3
```

Wszystkie zdarzenia można porównać na jednej osi czasu.

W Ubuntu 24.04 u Ciebie działa systemd-timesyncd. To prosty klient synchronizacji czasu. W nowszych wydaniach Ubuntu domyślna rola synchronizacji przesuwana jest w stronę chrony, ale w tym laboratorium nie zmienialiśmy działającego rozwiązania tylko po to, aby 'mieć nowsze'.

3. NTP - wykonane laboratorium i interpretacja wyników

Krok 1 - sprawdzenie stanu czasu:

```
timedatectl status
```

Zweryfikowany wynik:

```
Local time: sob 2026-10-10 21:03:14 CEST
Universal time: sob 2026-10-10 19:03:14 UTC
RTC time: sob 2026-10-10 19:03:14
Time zone: Europe/Warsaw (CEST, +0200)
System clock synchronized: yes
NTP service: active
RTC in local TZ: no
```

Interpretacja:

- Local time - czas lokalny wyświetlany użytkownikowi.
- Universal time - czas UTC.
- RTC - zegar sprzętowy komputera.
- System clock synchronized: yes - system potwierdza synchronizację.
- NTP service: active - mechanizm synchronizacji działa.
- RTC in local TZ: no - RTC jest utrzymywany w UTC, co jest typowe na Linuksie.

Krok 2 - sprawdzenie konkretnej usługi:

```
systemctl status systemd-timesyncd --no-pager
```

Zweryfikowano:

```
Active: active (running)
Status: "Contacted time server 185.125.190.58:123 (ntp.ubuntu.com)."
```

```
Initial clock synchronization ...
```

Port UDP 123 jest standardowo używany przez NTP.

Krok 3 - szczegółowe parametry synchronizacji:

```
timedatectl timesync-status
```

Parametr	Wynik	Znaczenie
Server	185.125.190.58 (ntp.ubuntu.com)	Aktualne źródło czasu
Stratum	2	Odległość logiczna od źródła referencyjnego
Offset	+4.063 ms	Różnica lokalnego zegara względem źródła
Delay	38.078 ms	Opóźnienie komunikacji
Jitter	1.700 ms	Zmienność kolejnych pomiarów
Poll interval	8 min 32 s	Odstęp między odpytywaniem serwera

Wniosek z laboratorium: synchronizacja działała prawidłowo; offset rzędu kilku milisekund był bardzo mały.

4. Logi - journald, journalctl i rsyslog

Linux może mieć kilka współpracujących warstw logowania. W naszym Ubuntu aktywne były zarówno systemd-journald, jak i rsyslog.

```
aplikacje / kernel / usługi
|
v
systemd-journald
|
+--> journalctl
|
`--> rsyslog
|
+--> /var/log/syslog
`--> zdalny serwer logów
```

systemd-journald zbiera zdarzenia w dzienniku systemd. journalctl jest narzędziem do ich przeglądania i filtrowania. rsyslog jest usługą, która może zapisywać logi do plików i przysyłać je dalej.

Wykonane testy:

```
systemctl status rsyslog --no-pager
```

Zweryfikowano: Active: active (running).

```
logger -t dzien26 "Test syslog - Dzień 26"

journalctl -t dzien26 --no-pager
```

Zweryfikowany wpis:

```
paź 10 21:07:22 mateusz-czarnik-ThinkCentre-M92P dzien26[5424]:
Test syslog - Dzień 26

grep "Test syslog - Dzień 26" /var/log/syslog
```

Zweryfikowany wpis w klasycznym pliku logu:

```
2026-10-10T21:07:22.437051+02:00
mateusz-czarnik-ThinkCentre-M92P
dzien26: Test syslog - Dzień 26
```

To potwierdziło, że komunikat można znaleźć zarówno przez journalctl, jak i w /var/log/syslog.

5. Centralny syslog w firmie

W małej sieci administrator może czasem sprawdzić logi lokalnie. W większej firmie to nie wystarcza. Urządzeń jest za dużo, a część logów może zniknąć po restarcie lub awarii urządzenia.

```
Router -----Switch ----- Firewall -----> Centralny serwer logów ---> wyszukiwanie / alarmy / audyt
Linux ----- /
VPN -----/
Windows -----/
```

Korzyści:

- jedno miejsce do analizy awarii;
- możliwość wyszukiwania po czasie, hoście, usłudze i treści;
- zachowanie logów nawet wtedy, gdy urządzenie źródłowe ulegnie awarii;
- łatwiejszy audyt bezpieczeństwa;
- możliwość integracji z systemem SIEM lub platforma observability.

W środowisku produkcyjnym warto rozważyć niezawodny transport TCP/TLS, retencję logów, kontrolę dostępu oraz synchronizację czasu na wszystkich źródłach logów.

6. SNMP - manager, agent, MIB i OID

SNMP (Simple Network Management Protocol) służy do monitorowania urządzeń i odczytywania ich parametrów. Typowy model składa się z systemu monitorującego oraz agenta działającego na urządzeniu.

```
Monitoring / NMS
|
| zapytanie SNMP
v
Agent SNMP na urządzeniu
|
+--> nazwa urządzenia
+--> uptime
+--> stan interfejsu
+--> liczniki ruchu
+--> błędy
`--> inne dane z MIB
```

Pojęcia:

- Manager / NMS - system monitorujący, np. Zabbix, LibreNMS, PRTG.
- Agent - proces na urządzeniu, który odpowiada na zapytania.
- MIB - uporządkowany opis obiektów, które można monitorować.
- OID - konkretny identyfikator obiektu w hierarchii MIB.

Przykład:

```
1.3.6.1.2.1.1.5.0 -> sysName
1.3.6.1.2.1.2.2.1.8 -> ifOperStatus
1.3.6.1.2.1.2.2.1.10 -> ifInOctets
```

```
1.3.6.1.2.1.2.2.1.16 -> ifOutOctets
```

Wersje SNMP - co trzeba wiedzieć jako junior

Wersja	Cechy	Ocena praktyczna
SNMPv1	Stara wersja, community string	Historyczna; nie wybieraj do nowego wdrożenia
SNMPv2c	Community string, wydajniejsze od v1	Popularne w labach i starszych sieciach, ale bez szyfrowania
SNMPv3	Użytkownicy, uwierzytelnianie, możliwość szyfrowania	Preferowane w nowych wdrożeniach

7. SNMP - wykonane laboratorium krok po kroku

Krok 1 - sprawdzenie obecności usługi:

```
systemctl status snmpd --no-pager
```

Początkowo otrzymaliśmy: Unit snmpd.service could not be found. To nie był błąd konfiguracji - agent po prostu nie był zainstalowany.

Krok 2 - sprawdzenie pakietów:

```
apt-cache policy snmpd snmp
```

Dostępna wersja pakietów: 5.9.4+dfsg-1.1ubuntu3.2.

```
sudo apt install snmpd snmp
```

Po instalacji:

```
systemctl status snmpd --no-pager
Active: active (running)
```

Krok 3 - sprawdzenie, gdzie agent nasłuchuje:

```
sudo ss -ltnp | grep ':161'
```

Zweryfikowany wynik:

```
127.0.0.1:161
[::]:161
```

To było bezpieczne ustawienie: agent był dostępny wyłącznie lokalnie, nie z całej sieci LAN.

Krok 4 - pierwszy odczyt SNMP:

```
snmpwalk -v2c -c public 127.0.0.1 1.3.6.1.2.1.1
```

Odczytano m.in.:

```
sysDescr -> Linux mateusz-czarnik-ThinkCentre-M92P ...
sysUpTime -> 0:10:53.19
sysName -> mateusz-czarnik-ThinkCentre-M92P
```

Domyślne wartości sysContact i sysLocation były przykładowe i nie reprezentowały prawdziwych danych.

Krok 5 - analiza aktywnej konfiguracji:

```
sudo grep -Ev '^[[:space:]]*(#|$)' /etc/snmp/snmpd.conf
```

Najważniejsze linie:

```
agentaddress 127.0.0.1,[::1]

view systemonly included .1.3.6.1.2.1.1
view systemonly included .1.3.6.1.2.1.25.1

rocommunity public default -V systemonly
rocommunity6 public default -V systemonly
rouser authPrivUser authpriv -V systemonly
```

Wniosek: community public miało tylko odczyt i było ograniczone do widoku systemonly.

Krok 6 - test niedozwolonej gałęzi:

```
snmpwalk -v2c -c public 127.0.0.1 1.3.6.1.2.1.2
```

Wynik:

```
No Such Object available on this agent at this OID
```

To potwierdziło, że widok systemonly faktycznie ogranicza dostęp.

Krok 7 - osobny, łatwy do cofnięcia plik laboratoryjny:

```
sudo tee /etc/snmp/snmpd.conf.d/day26-lab.conf > /dev/null <<'EOF'
view day26view included .1.3.6.1.2.1.1
view day26view included .1.3.6.1.2.1.2
rocommunity dzien26ro 127.0.0.1 -V day26view
EOF
```

Dlaczego tak? Zamiast edytować główny plik konfiguracyjny dodaliśmy osobny plik lab. To upraszcza rollback i dokumentację.

```
sudo systemctl restart snmpd
systemctl status snmpd --no-pager
```

Usługa po restarcie nadal działała poprawnie.

8. Monitoring interfejsów i liczników ruchu

Najpierw pobraliśmy opisy interfejsów:

```
snmpwalk -v2c -c dzien26ro 127.0.0.1 1.3.6.1.2.1.2.2.1.2
```

Zweryfikowane przykłady:

```
1 lo
2 eno1
3 virbr10
4 tailscale0
5 virbr0
6 virbr20
...
16 vnet0
17 vnet1
```

Stan operacyjny interfejsów:

```
snmpwalk -v2c -c dzien26ro 127.0.0.1 1.3.6.1.2.1.2.2.1.8
```

Najważniejsze wartości:

```
ifOperStatus = 1 -> up
ifOperStatus = 2 -> down
ifOperStatus = 3 -> testing
ifOperStatus = 4 -> unknown
ifOperStatus = 5 -> dormant
ifOperStatus = 6 -> notPresent
ifOperStatus = 7 -> lowerLayerDown
```

W naszym laboratorium:

```
eno1 -> indeks 2 -> ifOperStatus 1 -> UP
virbr20 -> indeks 6 -> ifOperStatus 2 -> DOWN
```

System monitoringu może porównywać te wartości cyklicznie i wygenerować alarm, gdy krytyczny interfejs zmieni stan z 1 na 2.

Liczniki ruchu:

```
snmpget -v2c -c dzien26ro 127.0.0.1 1.3.6.1.2.1.2.2.1.10.2 1.3.6.1.2.1.2.2.1.16.2
```

Pierwszy pomiar:

```
ifInOctets = 89,282,310
ifOutOctets = 1,614,832,769
```

Wygenerowano dodatkowy ruch:

```
ping -c 20 1.1.1.1 > /dev/null
```

Drugi pomiar:

```
ifInOctets = 90,009,395
ifOutOctets = 1,704,419,054
```

Wniosek: liczniki wzrosły. System monitoringu robi właśnie to samo: odczytuje licznik w dwóch punktach czasu, liczy różnicę, a następnie przelicza ją na ruch na sekundę i rysuje wykres.

Uwaga produkcyjna: klasyczne ifInOctets/ifOutOctets są licznikami 32-bitowymi. Na szybkich interfejsach w profesjonalnym monitoringu częściej wykorzystuje się 64-bitowe liczniki high-capacity, np. ifHCInOctets/ifHCOctets, aby ograniczyć problem szybkiego przepiętania licznika.

9. Bezpieczeństwo SNMP w firmie

SNMP jest bardzo użyteczne, ale źle wystawione może ujawniać dużo informacji o infrastrukturze. Dlatego produkcyjnie trzeba ograniczyć powierzchnię dostępu.

- Preferuj SNMPv3 z uwierzytelnianiem i szyfrowaniem (authPriv) zamiast v2c.
- Nie używaj community public jako produkcyjnego sekretu.
- Ogranicz źródła - tylko adresy serwerów monitoringu lub management VLAN.
- Stosuj read-only, jeśli monitoring nie potrzebuje zapisu.
- Ogranicz widok OID do tego, co system monitoringu naprawdę potrzebuje.
- Filtruj ruch firewalliem/ACL i nie wystawiaj UDP 161 do Internetu.
- Dokumentuj, który system ma prawo odczytywać które urządzenia.

Nasze laboratorium było bezpieczne, ponieważ rocommunity `dzien26ro` zostało ograniczone do 127.0.0.1 i do konkretnego widoku.

10. Skrypt diagnostyczny Bash - pełne omówienie

Utworzony plik:

```
~/dzien26-diagnostyka.sh
```

Uprawnienia:

```
-rwxrwxr-x ... /home/mateusz-czarnik/dzien26-diagnostyka.sh
```

Skrypt niczego nie zmienia. Tylko odczytuje stan i wykonuje testy.

Pełna treść skryptu:

```
#!/bin/bash

echo "=====
echo "  DIAGNOSTYKA SIECI - DZIEN 26"
echo "=====
echo "Data: $(date)"
echo "Host: $(hostname)"
echo

echo "[1] ADRESY IPv4"
ip -4 -br addr show scope global
echo

echo "[2] BRAMA DOMYSLNA"
GW=$(ip route | awk '/default/ {print $3; exit}')

if [ -n "$GW" ]; then
echo "Brama: $GW"

if ping -c 1 -W 2 "$GW" > /dev/null 2>&1; then
echo "OK - brama odpowiada"
else
echo "FAIL - brama nie odpowiada"
fi
else
echo "FAIL - brak bramy domyslniej"
fi
echo

echo "[3] DOSTEP DO INTERNETU"
if ping -c 1 -W 2 1.1.1.1 > /dev/null 2>&1; then
echo "OK - Internet po IP dziala"
else
echo "FAIL - brak odpowiedzi z 1.1.1.1"
fi
echo

echo "[4] DNS"
if getent ahostsv4 example.com > /dev/null 2>&1; then
echo "OK - DNS rozwiazuje example.com"
else
echo "FAIL - DNS nie rozwiazuje example.com"
fi
echo
```

```

echo "[5] NTP"
if [ "$(timedatectl show -p NTPSynchronized --value)" = "yes" ]; then
echo "OK - zegar jest zsynchronizowany"
else
echo "FAIL - zegar nie jest zsynchronizowany"
fi
echo

echo "[6] RSYSDLOG"
if systemctl is-active --quiet rsyslog; then
echo "OK - rsyslog działa"
else
echo "FAIL - rsyslog nie działa"
fi
echo

echo "[7] SNMP"
if systemctl is-active --quiet snmpd; then
echo "OK - snmpd działa"
else
echo "FAIL - snmpd nie działa"
fi
echo

echo "[8] TEST SNMP"
if snmpget -v2c -c dzien26ro \
127.0.0.1 \
1.3.6.1.2.1.1.5.0 > /dev/null 2>&1; then
echo "OK - lokalne zapytanie SNMP działa"
else
echo "FAIL - lokalne zapytanie SNMP nie działa"
fi

echo
echo "=====
echo " KONIEC DIAGNOSTYKI"
echo "=====

```

Co robi każda część?

Fragment	Znaczenie praktyczne
#!/bin/bash	Wskazuje interpreter Bash.
date / hostname	Dodaje czas i nazwę hosta do raportu.
ip -4 -br addr	Pokazuje adresy IPv4 w krótkiej formie.
ip route + awk	Pobiera pierwszą bramę domyślną do zmiennej GW.
ping do bramy	Sprawdza podstawową łączność lokalną.
ping 1.1.1.1	Sprawdza łączność IP z Internetem bez zależności od DNS.
getent ahostsv4 example.com	Sprawdza rozwiązywanie nazw DNS.
timedatectl show ...	Sprawdza, czy zegar jest zsynchronizowany.
systemctl is-active	Sprawdza stan konkretnej usługi bez dużej ilości tekstu.
snmpget ... sysName	Sprawdza nie tylko proces snmpd, ale rzeczywistą odpowiedź SNMP.
> /dev/null 2>&1	Ukrywa standardowy output i błędy; liczy się kod powodzenia testu.

Zweryfikowany wynik uruchomienia skryptu:

```

[1] ADRESY IPv4
eno1 UP 192.168.0.51/24
tailscale0 UNKNOWN 100.94.93.66/32
virbr0 UP 192.168.122.1/24
...

[2] BRAMA DOMYSLNA
Brama: 192.168.0.1
OK - brama odpowiada

[3] DOSTEP DO INTERNETU

```

OK - Internet po IP działa

[4] DNS

OK - DNS rozwiązuje example.com

[5] NTP

OK - zegar jest zsynchronizowany

[6] RSYSLLOG

OK - rsyslog działa

[7] SNMP

OK - snmpd działa

[8] TEST SNMP

OK - lokalne zapytanie SNMP działa

Stan `UNKNOWN` przy tailscale0 nie oznacza automatycznie awarii. Interfejsy tunelowe nie zawsze mają taki sam model stanu linku jak fizyczny Ethernet.

11. Praktyczny schemat diagnostyczny juniora

Ważniejsza od liczby poznanych komend jest kolejność rozumowania. Dobry junior nie zaczyna od przypadkowych zmian.

```
OBJAW
|
v
1. Czy host ma poprawny adres?
ip -4 -br addr
|
v
2. Czy jest brama?
ip route
|
v
3. Czy brama odpowiada?
ping <gateway>
|
v
4. Czy działa łączność po IP?
ping 1.1.1.1
|
v
5. Czy działa DNS?
getent / dig / nslookup
|
v
6. Czy konkretna usługa działa?
systemctl / ss / Test-NetConnection
|
v
7. Czy logi mówią coś więcej?
journalctl / syslog
|
v
8. Dopiero wtedy naprawa i ponowna weryfikacja
```

Dlaczego osobno testujemy Internet po IP i DNS?

1.1.1.1 działa + nazwa nie działa
=> warstwa IP/routing prawdopodobnie działa
=> problem zawężamy do DNS

1.1.1.1 nie działa
=> problem jest wcześniej:
adresacja / brama / routing / firewall / łącze

Uwaga terminologiczna: ping używa ICMP i sprawdza łączność na poziomie warstwy sieciowej, a nie warstwy transportowej.

12. Bonus - rzeczywista diagnostyka RDP/VM z początku Dnia 26

Przed rozpoczęciem lekcji pojawił się komunikat:

```
nc: connect to 192.168.122.24 port 3389 (tcp) failed: No route to host
```

Zamiast od razu zmieniać RDP lub firewall wykonaliśmy jeden test:

```
virsh domstate windows-server-2025
```

Wynik:

```
off
```

Naprawa:

```
virsh start windows-server-2025
```

Weryfikacja:

```
virsh domstate windows-server-2025
running

nc -vz 192.168.122.24 3389
Connection to 192.168.122.24 3389 port [tcp/ms-wbt-server] succeeded!
```

Wniosek: przyczyną nie był RDP, tylko wyłączona maszyna wirtualna. To bardzo dobry przykład zasady: objaw -> hipoteza -> jeden test -> naprawa -> weryfikacja.

13. REST API, JSON, Ansible, Terraform i AI - krótkie wprowadzenie

To rozszerzenie Dnia 26. Nie wykonywaliśmy pełnego laboratorium, ale warto rozumieć, gdzie te technologie pojawiają się w nowoczesnej administracji.

REST API i JSON

```
GET /api/interfaces/ether1

{
  "interface": "ether1",
  "status": "up",
  "ip": "192.168.0.1"
}
```

API pozwala programowi pobrać lub zmienić dane bez klikania w GUI. JSON jest popularnym formatem danych przesyłanych przez API.

Ansible

Ansible pomaga wykonywać tę samą konfigurację na wielu systemach w sposób powtarzalny. Przykład: wdrożenie użytkownika, pliku konfiguracyjnego lub ustawienia na wielu serwerach.

Terraform

Terraform opisuje infrastrukturę jako kod. Jest szczególnie popularny przy chmurze i tworzeniu zasobów infrastrukturalnych. Nie zastępuje wszystkich narzędzi konfiguracji - często współpracuje z innymi rozwiązaniami.

AI w administracji

AI może pomagać w analizie logów, podsumowaniu incydentu, tworzeniu szablonów skryptów i dokumentacji. Nie powinno się jednak bez weryfikacji uruchamiać wygenerowanych zmian w produkcji.

14. Co warto umieć powiedzieć na rozmowie rekrutacyjnej?

Pytanie	Krótką dobrą odpowiedź
Po co NTP?	Żeby urządzenia miały zgodny czas; jest to ważne dla logów, uwierzytelniania i audytu.
Co robi journalctl?	Odczytuje i filtruje logi z systemd journal.
Co robi rsyslog?	Zbiera/routuje logi, zapisuje je do plików i może wysyłać je do zdalnego serwera.
Co to SNMP?	Protokół do monitorowania i zarządzania urządzeniami; monitoring odpytuje agenta o konkretne OIDy.
Co to OID?	Unikalny identyfikator konkretnego obiektu w MIB, np. nazwy hosta lub stanu interfejsu.
Dlaczego SNMPv3?	Daje uwierzytelnianie i może zapewnić szyfrowanie; v2c używa community stringów.

Pytanie	Krótką dobrą odpowiedź
Co oznacza ifOperStatus=1?	Interfejs jest operacyjnie UP.
Jak liczysz ruch z SNMP?	Biorę dwa odczyty licznika, liczę różnicę i dzielę przez czas między pomiarami.
Po co osobny test IP i DNS?	Żeby rozdzielić problem łączności/routingu od problemu rozwiązywania nazw.

Wynik quizu Dnia 26

5/6 punktów (~83%). Do powtórki: `ifOperStatus = 1` oznacza `up` oraz precyzyjne rozróżnienie testu łączności IP od testu DNS.

15. Checklisty, rollback i następne kroki

Checklist - NTP

- `timedatectl status` -> System clock synchronized = yes
- NTP service = active
- `systemctl status systemd-timesyncd`
- `timedatectl timesync-status` -> sprawdź server, stratum, offset, delay, jitter

Checklist - logi

- `systemctl status rsyslog`
- `logger -t test "wiadomość"`
- `journalctl -t test`
- `grep` wiadomości w `/var/log/syslog`

Checklist - SNMP

- `systemctl status snmpd`
- `ss -ltnp | grep ':161'`
- `snmpwalk/snmpget` dla konkretnego OID
- ogranicz źródła, community/użytkowników i widoki
- produkcja: preferuj SNMPv3 i segment zarządzający

Rollback dodatkowej konfiguracji laboratoryjnej SNMP

```
sudo rm /etc/snmp/snmpd.conf.d/day26-lab.conf
sudo systemctl restart snmpd
```

To usuwa tylko naszą dodatkową konfigurację lab. Nie odinstalowuje pakietów.

Co zostawiliśmy po Dniu 26?

- `snmpd` i `snmp` są zainstalowane;
- agent nadal nasłuchuje lokalnie na `127.0.0.1 / ::1`;
- plik `/etc/snmp/snmpd.conf.d/day26-lab.conf` zawiera dodatkowy widok lab;
- skrypt `~/dzien26-diagnostyka.sh` pozostaje jako materiał do portfolio;
- NTP i `rsyslog` pozostają działającymi usługami systemowymi.

Sugestia do portfolio GitHub

```
docs/
day26-monitoring.md

scripts/
dzien26-diagnostyka.sh

snmp/
day26-lab.conf.example
```

```
screenshots-or-results/  
ntp-timesync-status.txt  
snmp-interface-status.txt  
diagnostics-output.txt
```

W README warto opisać nie tylko komendy, ale też co było celem, jak zweryfikowano wynik i jak cofnąć zmianę.

Następny krok programu

Dzień 27: scalenie projektu małej firmy - użytkownicy, serwery i goście w osobnych segmentach, DNS/DHCP, routing, firewall, VPN, plan adresacji i dokumentacja.

Źródła i aktualna dokumentacja

Poniższe źródła wykorzystano do weryfikacji informacji technicznych. Laboratorium i konkretne wyniki pochodzą z wykonanych przez Ciebie ćwiczeń.

Źródło	Adres
Ubuntu Server - Synchronize time using timedatectl and timesyncd	https://ubuntu.com/server/docs/how-to/networking/timedatectl-and-timesyncd/
Ubuntu Server - About time synchronization	https://ubuntu.com/server/docs/explanation/networking/about-time-synchronisation/
Ubuntu Noble manpage - systemd-timesyncd	https://manpages.ubuntu.com/manpages/noble/man8/systemd-timesyncd.8.html
rsyslog official documentation - Getting Started	https://docs.rsyslog.com/doc/getting_started/index.html
Net-SNMP - snmpd.conf	https://www.net-snmp.org/docs/man/snmpd.conf.html
Net-SNMP - Interfaces MIB / ifOperStatus	https://www.net-snmp.org/docs/mibs/interfaces.html
Net-SNMP - IF-MIB high-capacity counters	https://www.net-snmp.org/docs/mibs/ifMIBObjects.html

Podsumowanie końcowe

Po Dniu 26 potrafisz sprawdzić synchronizację czasu, wygenerować i znaleźć wpis w logach, uruchomić lokalnego agenta SNMP, ograniczyć jego widok, odczytać stan interfejsu i liczniki ruchu, a także wykonać podstawową automatyczną diagnostykę sieci skrypcem Bash. Najważniejszy nawyk to nie sama znajomość komend, lecz uporządkowana diagnostyka: objaw -> hipoteza -> jeden test -> interpretacja -> naprawa -> weryfikacja.